Retrievers

A retriever is a component (Runnable) that accepts a string (query) as input and returns a list of Document object as output.

Role in RAG: It acts as the search engine for our application. It scans a data source (Vector Store, API, etc.) to fetch relevant information before the LLM generates an answer.

⇒ Key Distinction: While a vector store is a database, a Retriever is the interface/logic used to fetch data. A vector store does simple similarity search; a Retriever can employ complex strategies (like re-ranking or query expansion).

Retrievers can be organized into two different categories:

(a) By Data Source: Where the data comes from (eg., WikipediaRetriever, ArxivRetriever, VectorStoreRetriever).

(b) By Search Strategy: How the data is found (e.g. MultiQuery Retriever, MMR, Contextual CompressionRetriever).

(1). Vector Store Retriever (Standard) ⇒ It performs standard semantic similarity search (Cosine Similarity). It is fast & simple. Although it can return redundant information if multiple documents contain nearly identical text.

(2). Maximum Marginal Relevance (MMR) ⇒ Standard search often returns "Redundant" results. However, MMR selects documents that are relevant to the query but dissimilar to each other. It picks the best match first, then penalizes subsequent documents that are too similar to the first one.

* lambda_mult: This parameter controls diversity. The value ranges between 0 & 1. 1.0 ⇒ Pure similarity search (Standard search)
0.0 ⇒ Maximum diversity

(3). Multi-Query Retriever ⇒ User queries are often ambiguous (e.g. "How to stay healthy?" could mean diet, exercise or mental health) A single vector search might miss the user's intent.

Scanned with OKEN Scanner

MultiQuery Retriever uses an LLM to generate multiple variation's of the users query. Then it retrievers documents for all the variations. Takes the union of all unique documents to provide a comprehensive answer. Thus, it increases the recall (likelihood of finding the right answer) for vague queries.

(4). Contextual Compression Retriever A document can be relevant but 90% of its content is irrelevant noise (e.g. a paragraph about "Photosynthesis") inside a document mostly about the "Grand Canyon" Sending the whole document wastes tokens and confuses the LLM.

Contextual Compression Retriever consists of –

(a) Base Retriever: Fetches initial documents.

(b) Compressor (LLM): Reads the document and the query, then deletes irrelevant sentence/ paragraphs.

It returns only the specific lines relevant to the answer, drastically reducing context size

(5). Ensemble Retriever The Ensemble Retriever allows us to combine a Keyword Search (BM25) and a Semantic Search (Vector) simultaneously. Keyword search is better for exact matches (e.g. Product IDs, specific names), while Vector search is better for concepts. Ensemble combines them using an algorithm called Reciprocal Rank Fusion (RRF) to get the best of both worlds.

(6). Self-Querying Retriever It solves the "metadata filtering" problem. If a user asks, "Show me sci-fi movies from 1990", a standard vector search might return sci-fi movies from 2020 because they are semantically similar. The Self-Query Retriever uses an LLM to split the query into two parts: (a) Vector component: "sci-fi" movies for similarity search.

(b) Filter component: "year == 1990" (structured metadata filter)

(7). Parent Document Retriever It solves the "chunk size" dilemma. Small chunks are better for accurate search (vectors), but large chunks are better for the LLM to understand context.

Scanned with OKEN Scanner

⇒ We index small chunks for the search, but when a match is found, the retriever returns the Parent Document (the larger original chunk) to the LLMs. This gives us precise retrieval over full context.

Scanned with OKEN Scanner